

VestigeKV: An Implementation White Paper

Training-free sparse attention for NoPE-MLA caches —
a from-zero guide for operator engineers

VestigeKV project

2026-09-12 (v2, English; algorithm spec frozen against `sglang` port commit `e44b914a7`,
branch `vestigekv`)

Abstract

This document specifies, at implementation depth, the VestigeKV sparse attention for NoPE-MLA key-value caches: the mathematics of the selection signal, every data structure, every per-step algorithm, the contract the fused attention kernel must satisfy, and the validation protocol. It is written so that an operator engineer holding only this document and stock model weights can reproduce the method from zero on a new hardware platform (GPU, NPU, or other accelerators). Everything stated as measured comes from the pre-registered experimental record of the VestigeKV project; everything stated as a contract was validated in a production `sglang` port. No training, no quantization, no weight or kernel-math changes are involved: the method is purely a change of *which rows attention reads*.

Contents

1	Scope and preconditions	2
1.1	Deliverable scope	2
1.2	Architectural preconditions	2
1.3	Mathematical soundness of frozen selection under NoPE	3
2	The selection signal (tier 1)	3
2.1	Definition	3
2.2	Selection rule	4
3	Data structures and memory contract	5
3.1	Index width: int64 today, int32 as a documented lever	5
4	The decode path	6
4.1	Single-request (batch 1): in-graph $O(1)$ append	6
4.2	Batched decode: out-of-graph vectorized repack	6
4.3	Prefill and lifecycle	7
5	The attention-kernel contract	7
6	The recall tier (tier 2, the standard configuration)	8
6.1	The selection principle: certified competition, and why the fired set is variable	8
6.2	Index build (per layer; provisional at the first decode step, calibrated asynchronously)	9
6.3	Per-step query (each decode step; per layer)	11

6.4	Fused in-graph implementation (the shipped scan)	11
6.5	How the per-step recall count arises	12
6.6	Is tier 1 still necessary at long context, and does recall depend on it?	12
6.7	Relation to ArkVale	13
6.8	The fetch cap: an optional deployment optimization (not part of the algorithm)	13
7	Parameters	13
8	Decode-time block close and rebalance	14
9	Cost model and validation targets	15
10	Validation protocol	16
11	Observed defect classes and mitigations	17

1 Scope and preconditions

1.1 Deliverable scope

A drop-in replacement for the *decode-time* attention of a NoPE-MLA transformer layer. Prefill attention, all projections, all other layers, and the sampling head are untouched. The replacement attends, per decode step, over a small selected subset of the KV cache instead of all T rows:

$$\text{attended}(t) = \underbrace{\text{kept}(t)}_{\text{rebalanced at each block close}} \cup \underbrace{\text{tail}(t)}_{\text{unclosed rows, incl. decoded so far}} \cup \underbrace{\text{recalled}(t)}_{\text{standard, per-step}}$$

with $|\text{kept}| \approx \rho T$ ($\rho = 1/32$ recommended). Evicted rows are *never deleted*: they move, bit-exact, to an archive reachable by the recall tier (Section 6; the standard configuration, necessary at long context). The deployed invariant is a *partition* of the cache, not a truncation — and the partition is only meaningful if the archive is consulted, so **the recall tier is required implementation scope**: a port that implements tier 1 alone is an incomplete state of this method, not a configuration of it, and must not expose a switch that disables recall (the only recall knob is the explicit fetch cap).

1.2 Architectural preconditions

1. **MLA (multi-head latent attention) cache.** The per-token cache row is a single latent vector shared by all heads (MQA-style single KV head). For Kimi Linear 48B: row width $576 = 512 + 64$, layout

$$\tilde{C}_u = [\hat{c}_u \in \mathbb{R}^{512} \mid r_u \in \mathbb{R}^{64}],$$

where $\hat{c}_u$ is the RMS-normalized content latent (*also the value*: V is read from the first 512 dims) and r_u is the decoupled “rope branch”.

2. **NoPE: the branch is never rotated.** In the target model the rotary call is absent (`mla_use_nope` in the config); r_u is cached as produced. This is the load-bearing precondition: under RoPE the identical operator collapses (measured $0.89 \rightarrow 0.08$ needle retrieval at $32\times$), because the selection signal of Section 2 exists only when training has repurposed the positional channel. *A port onto a RoPE model is unsupported; the collapse is a measured result, not a hypothesis.* The backend enforces this itself: registration refuses any MLA config

that is not NoPE (and refuses `page_size≠1` and speculative decoding), raising rather than degrading silently.

3. **bf16 cache rows, never quantized.** Both tiers keep the 576-dim latents bit-exact, so every quality effect attributes to selection and scheduling (a controlled-variable rule). The scan’s *index metadata* is stored 16-bit (side bf16, sketch fp16), calibrated after rounding; a port that quantizes the cache rows themselves has left the validated envelope.

1.3 Mathematical soundness of frozen selection under NoPE

Write one decode step’s attention over cache rows $\{\tilde{C}_u\}_{u < t}$ with expanded query q_t (after the standard MLA absorb):

$$o_t = \sum_u \text{softmax}_u(s q_t \cdot \tilde{C}_u) \hat{c}_u.$$

Under NoPE both the score map $u \mapsto q_t \cdot \tilde{C}_u$ and the value map $u \mapsto \hat{c}_u$ depend on u *only through the row content* $\tilde{C}_u$ — no term depends on the position index itself.

Property 1 (Permutation/multiset invariance). *The NoPE-MLA attention output is a function of the multiset $\{\{\tilde{C}_u\}\}$ of cache rows (plus the query). Reordering, re-timing, or re-addressing rows never changes o_t .*

Corollary 1 (Ranking stationarity). *Any selection rule that is a function of row contents only can be computed at any time after the rows exist and never goes stale: a kept set chosen at prefill end is exactly the set the same rule would choose at any later step over the same rows. Frozen rankings are exact, not approximate.*

Two engineering consequences follow directly and shape the whole implementation: (a) selection runs *off* the decode critical path — the implementation recomputes it at prefill end and at every block close (every 4096 decoded tokens), where the kept quota $m = \text{round}(\rho \cdot \text{closed})$ grows proportionally and the global top- m is rebalanced over all closed rows (rows may move in either direction: a long-kept row can be demoted to the archive); (b) between closes the per-step sparse pattern is a *constant* of the decode graph, so it is compatible with graph-capture execution (CUDA graphs and their equivalents) at zero per-step host cost. Under RoPE neither holds: scores depend on the query-key *distance*, so any frozen pattern goes stale at the rate of the rotary frequency bands.

2 The selection signal (tier 1)

2.1 Definition

Selection reads *only* the 64-dim branch slice r_u (11% of the row; one contiguous read). The cache prefix is processed in fixed, non-overlapping blocks of $B = \text{CLOSE_BLOCK} = 4096$ rows (only full blocks are scored; the unclosed tail is always attended and needs no score). Within one block, stack the branches $r \in \mathbb{R}^{B \times 64}$ along the sequence axis and score each row by its residual against a low-pass reconstruction along that axis:

$$\sigma_u = \|r_u - (\mathcal{F}^{-1} \mathbb{K}_{[0, \kappa]} \mathcal{F} r)_u\|_2, \quad \kappa = 16, \tag{1}$$

where $\mathcal{F}$ is the real FFT *within the block* (length B , never the whole prefix), applied independently to each of the 64 channels, $\mathbb{K}_{[0, \kappa]}$ zeroes all but the lowest κ frequency bins, and $\mathcal{F}^{-1}$ inverts at

length B . The fixed block window is load-bearing: κ counts frequency bins, and bin k of a length- B transform corresponds to period B/k — a whole-prefix transform would let the cutoff drift with context length (≥ 256 tokens at $B=4096$ vs. $\geq 32k$ tokens at $T=512k$). Per-block transforms keep σ scale-invariant, incremental (each block is transformed exactly once, at its close, and σ is immutable thereafter), and bit-identical to the reference policy. Intuition: under NoPE, training repurposes the positionless branch into a *salience channel*; rows that carry retrieval-critical content are spectral anomalies against the smooth background. The signal is query-independent — this is what Corollary 1 needs.

2.2 Selection rule

Over the closed rows only (closed = $\lfloor T/B \rfloor \cdot B$):

$$\text{keep} = \text{top-}m(\sigma_{\text{closed}}) \cup \{0, \dots, \text{sinks} - 1\}, \quad m = \max(1, \text{round}(\rho \cdot \text{closed})), \quad (2)$$

with sinks = 4 (always keep the first rows). There is *no* fixed recent window: the unclosed tail $\{\text{closed}, \dots, T - 1\}$ ($< B = 4096$ rows, including every token decoded since the last close) is attended unconditionally and absorbs that role. Top- m is a single *global* selection over the whole closed prefix, re-run at every block close with the growing quota m (a proportional- m rebalance, licensed by Corollary 1). The rule is applied per layer, independently; only the full-attention (MLA) layers participate (in Kimi Linear 48B, 7 of 27 layers; the linear-attention layers have constant-size state and are untouched).

Algorithm 1 Tier-1 selection at prefill end / block close (per MLA layer)

- 1: $r \leftarrow \text{pool_rows}[\text{slots of this request}][:, 512:576]$ ▷ contiguous slice, $[T, 64]$
 - 2: **for** each full 4096-row block b newly closed since the last selection **do**
 - 3: $f \leftarrow \text{rfft}(r[b].\text{float}(), \text{dim} = 0)$; $f[\kappa:] \leftarrow 0$; $\ell_b \leftarrow \text{irfft}(f, n=4096, \text{dim} = 0)$
 - 4: $\sigma[b] \leftarrow \|r[b] - \ell_b\|_2$ over dim - 1 ▷ per-block, immutable once computed
 - 5: keep $\leftarrow$ boolean mask from Eq. (2) over all closed rows ▷ global top- m ,
 $m = \max(1, \text{round}(\rho \cdot \text{closed}))$
 - 6: write kept_buf[slot, 0:n] $\leftarrow$ pool indices of keep rows followed by the unclosed tail (ascending);
kept_len[slot] $\leftarrow n$
 - 7: invalidate any per-slot decode-side state cached for a previous occupant of this slot ▷
slot-reuse; see Section 11
-

Numerical notes: compute σ in fp32 (the branch is bf16 in the pool); the rFFT length is the block length 4096, never a padded or whole-prefix length; the production path is a fused Triton kernel computing the same statistic in its exact projection form ($\sigma_u = \|R_u - C(C^\top R)_u\|$ over a fixed trigonometric basis C of the block, fused with a radix-histogram pass for the global top- m) — mathematically identical to the rFFT chain, $\sim 28\times$ faster at batched prefill; ties in top- m may break arbitrarily (any tie-break is valid by Property 1).

Implementation path. The production path is the fused Triton kernel `sigma_fused`: the rFFT→low-pass→irFFT chain is evaluated in its exact closed form — an orthogonal projection onto the $(2\kappa - 1)$ -dim trigonometric subspace of the fixed block, so $\sigma_u = \|R_u - c_u^\top (C^\top R)\|$ with a precomputed basis C — fused with a shared radix histogram (`atomic_add` on the fp32 bit-pattern’s high bits) that feeds an exact top- m selection (`topm_from_hist`: reverse-cumsum the histogram to the boundary bin, then `topk` inside it). One launch per block set, $\sim 28\times$ faster than the batched-`torch.fft` chain

at prefill, and bit-comparable (fp32-ieee accumulations; the projection form differs from cuFFT by $\sim 10^{-7}$ rms against a 1.2×10^{-3} bf16 input quantization floor, and measured top- m agreement is 100%). The stock `torch.fft` / `torch.topk` chain remains in-tree as the reference oracle — a registered test asserts the fused kernel matches it exactly. Either way this path runs at prefill end and at each block close — once per 4096 generated tokens per request, off the per-token decode path. Two disciplines are preserved in both forms: (i) one numerical flavour of σ per block — never mix two implementations of the same statistic into one global top- m ; (ii) σ is computed once per block and never recomputed. The seven fused Triton kernels of Section 6.5 are the *decode-hot-path* recall step (per token, per layer); the selection path is a different regime.

3 Data structures and memory contract

All structures are per *(request-slot, layer)*; a “slot” is the row index in the runtime’s request-to-token table.

name	shape / dtype	written at	read at
latent pool	$[N_{\text{pool}}, 576]$ bf16	every token append	attention kernel
r2t	$[\text{max_reqs}, \text{ctx}]$ int	runtime-owned	prefill selection
kept_buf	$[\text{max_reqs}, \text{ctx}]$ int	prefill; +1/step	CSR build
kept_len	$[\text{max_reqs}]$ int64	prefill; +1/step	CSR build
CSR (indptr, indices)	$[B+1], [\sum n_i]$ int	every decode step	attention kernel

CSR contract (this is what the kernel consumes). For batch entry i , the kernel reads rows `indices[indptr[i] : indptr[i+1]]` as absolute pool indices. Boundaries are shared ($\text{end}_i = \text{start}_{i+1}$), so the packed indices must be contiguous with no gaps. Row order within a request is irrelevant (Property 1) — keep ascending order anyway for coalescing.

Pad-row convention. Graph-capture runtimes pad the batch to a captured bucket size. Every padded CSR slot must point at exactly one reserved pad row (pool row 0) so its softmax is non-empty; the padded lanes’ outputs are discarded by the runtime. Never let a padded lane read another request’s rows and never leave it an empty range.

Growth. `kept_len` grows by exactly 1 per decode step (the just-written token’s pool slot is appended — the “tail”). Size `kept_buf`’s second dim to the context limit; overflow is then impossible by construction.

3.1 Index width: int64 today, int32 as a documented lever

All slot-id storage (`kept_buf`, the fetch buffers, the CSR indices) is int64 in the shipped implementation. That width is *inherited*, not needed: the buffers take the dtype of the runtime’s `kv_indices` metadata for drop-in compatibility, while the runtime’s own authoritative slot map (`r2t`) is int32. The value domain makes int64 pure overhead — a pool row costs $576 \times 2\text{B}$ per MLA layer, so $\text{pool_rows} < 2 \times 10^7$ on any realistic GPU, two orders of magnitude below 2^{31} ; int32 cannot overflow.

The cost of the wider word, for Kimi Linear 48B (7 MLA layers) at a 512k context cap with 64 request slots and a batch-32 graph:

buffer	shape	int64	int32
kept table	[7, 65, 524288]	1.9 GB	0.95 GB
graph CSR indices	[7, $32 \times 524288 + 1$]	0.94 GB	0.47 GB
fetch buffers	[7, 65, 4096]	15 MB	7 MB
total per node		2.9 GB	1.4 GB saved

The compute side of the lever is negligible by design: the recall path moves only 4-to-8-byte slot ids, $\sim 1/288$ of the latent-row traffic, so halving the index width buys memory, not speed. Conversion is mechanically simple whenever wanted: `scatter_/_gather_` require only the index *argument* to be int64 (the stored values may be int32, and the boundary casts already exist), and the Triton pack/scan kernels promote every loaded index with an explicit `.to(tl.int64)`, so they recompile against int32 buffers unchanged. Length and indptr arrays are kilobyte-scale and stay as they are. We record the lever here rather than pull it: the reported system keeps the runtime’s native index dtype, and the memory figures in Section 9 are therefore conservative.

4 The decode path

4.1 Single-request (batch 1): in-graph $O(1)$ append

With batch 1 the CSR is two scalars, and the per-step state change is one append. Both are expressible as fixed-shape tensor ops on fixed-address buffers, so they can be *recorded inside the capture* and replayed with zero host work:

Algorithm 2 Per-step in-graph refresh, batch 1 (all MLA layers)

- 1: $\text{slot} \leftarrow \text{req_pool_indices}[0:1]$; $\text{loc} \leftarrow \text{out_cache_loc}[0:1]$
 - 2: **for** each local MLA layer ℓ **do**
 - 3: $n \leftarrow \text{kept_len}_\ell.\text{gather}(0, \text{slot})$
 - 4: $\text{flat} \leftarrow \text{slot} \cdot \text{cap} + n$
 - 5: $\text{kept_buf}_\ell.\text{view}(-1).\text{scatter_}(0, \text{flat}, \text{loc})$
 - 6: $\text{kept_len}_\ell.\text{scatter_}(0, \text{slot}, n+1)$
 - 7: $\text{indptr}_\ell[0:1] \leftarrow \text{slot} \cdot \text{cap}$; $\text{indptr}_\ell[1:2] \leftarrow \text{flat} + 1$
-

The kernel is then bound (at capture time) to $\text{indices} = \text{kept_buf}_\ell.\text{view}(-1)$ and $\text{indptr} = \text{indptr}_\ell$: with batch 1 the request’s rows are already contiguous in its own `kept_buf` row, so no packing is needed at all.

Capture-safety rule (portable). Inside a capture region, never extract a scalar from device memory (`t[0]`-style indexing performs a hidden device→host sync and invalidates stream capture on CUDA; the analogous restriction exists on other platforms’ graph capture). Use 1-D gather/scatter/copy with tensor indices only. This restriction is a hard rule for all graph-capture platforms; the reference port records the corresponding capture-invalidation failure.

4.2 Batched decode: out-of-graph vectorized repack

With batch > 1 , boundaries are shared so a contiguous pack is required each step. Run it *outside* the captured region (in the runtime’s per-step metadata-refresh hook), fully vectorized — a per-request loop of small ops costs milliseconds at batch 16 and was measured to erase the method’s gain:

Algorithm 3 Per-step out-of-graph refresh, batch B padded to bucket B_g (per MLA layer)

```

1: slots  $\leftarrow$  req_pool_indices[0: $B$ ]  $\triangleright B$  real requests; out_cache_loc has exactly  $B$  entries
2:  $n \leftarrow$  kept_len.gather(0, slots); append loc at (slots  $\cdot$  cap +  $n$ ); lens  $\leftarrow n+1$ ; scatter back
3:  $k_{\max} \leftarrow \max(\text{lens})$   $\triangleright$  one small D2H sync
4: grid  $\leftarrow$  kept_buf.view(-1).gather(0, (slots[:, None]  $\cdot$  cap + arange( $k_{\max}$ )[None, :]))
5: valid  $\leftarrow$  arange( $k_{\max}$ )[None, :] < lens[:, None]; total  $\leftarrow$  sum(lens)  $\triangleright$  second D2H sync
6: indices[0:total]  $\leftarrow$  grid[valid]  $\triangleright$  masked select = the pack
7: pad slots  $B..B_g-1$ : one pad row each (row 0); indptr  $\leftarrow$  [0, cumsum(lens), total+1, ...]

```

Padding trap (measured crash). The runtime may pad seq_lens/req_pool_indices to the captured bucket while out_cache_loc keeps only the real entries. Derive the real batch from out_cache_loc’s length, never from the padded shape; indexing the padded lanes overruns.

4.3 Prefill and lifecycle

1. **Short-context dense fallback** (deployment option, default on at 32 768 tokens = 8 close blocks, SGLANG_VESTIGEKV_ACTIVATION_MIN_TOKENS; 0 disables): below the threshold selection is off, attended = everything, byte-identical to stock decode. The threshold exists because short context is where the method is at a *disadvantage*: the recall pipeline costs a near-flat ≈ 0.1 ms/step regardless of context, while dense attention below ≈ 32 k is cheaper than kept+scan+fetch, so compression has nothing to buy there and only the fixed cost remains. On the crossing step the whole prefix closes in one pass — blockwise σ is immutable once computed and the global top- m rebalance commutes with closing order, so the kept set equals compressing from token 0. **Every experiment reported here runs with the fallback disabled** (threshold 0): the compressed path is what gets measured at every length, so the short-context numbers below include the regime where the method pays its fixed cost for no gain — they are the honest worst case, not the tuned one.
2. **Chunked prefill**: run Algorithm 1 at each chunk end (or once at prefill end); the last write wins and is the frozen kept set.
3. **Decode**: Algorithms 2/3 per step; the fused kernel runs unmodified over fewer rows.
4. **Slot reuse**: a new request occupying a slot must not see any predecessor state — invalidate at prefill (see Section 11).

5 The attention-kernel contract

VestigeKV *reuses* the platform’s fused MLA decode kernel; nothing in the kernel math changes. The port therefore reduces to guaranteeing that the target platform’s kernel satisfies the following contract:

1. **Indexed gather**: keys/values are fetched by absolute pool indices from the CSR (Section 3); rows are *not* contiguous in the pool. On the validated GPU port the scattered gather cost is within the measurement noise; on platforms with strong contiguity requirements, stage the gather through scratch memory.
2. **MLA absorb decode form**: single shared KV head; per-head query $q = [q_{\text{nope}} W_{kc} \mid q_{\text{rope}}] \in \mathbb{R}^{576}$; score = $s q \cdot \tilde{C}_u$ with $s = d^{-1/2}$ per the model config; value = $\tilde{C}_u[0:512]$; output

re-projected by W_{vc} outside the kernel. (This is stock MLA; a platform already serving DeepSeek-style MLA decode satisfies this item.)

3. **Split-KV two-stage reduction:** stage 1 computes partial softmax accumulators + log-sum-exp (LSE) per KV split; stage 2 merges by LSE. *Required behavior:* empty splits (zero rows) merge out as $-\infty$ LSE. This is what makes a grid sized for the full context correct when the CSR shrinks the row count $32\times$: extra splits are empty, not wrong. Do not resize the grid per step in graph mode; rely on empty-split correctness.
4. **Graph capture:** the kernel must read `indptr` / indices *contents* at execution time from fixed addresses (the buffers of Section 3), so a replayed graph sees each step’s refreshed values. Any metadata precomputed at capture into registers/immediates breaks the scheme.
5. **Splice for recall:** the fetched rows join as one extra split-KV partition (merged by the same LSE path); no kernel change. Graph capture requires a *fixed fetch-buffer width* W , not a cap: the algorithm is uncapped, and a safety width (default 4096 > the observed worst fire of 3,983) serves its semantics unchanged inside the graph, with overflow surfaced by telemetry. The cap is merely one policy for choosing W .

Non-requirements (for tier-1 attention): the attention over the kept+-fired rows needs no new kernel, no custom softmax, no changes to W_{kc}/W_{vc} , no rope handling (there is none under NoPE), and no quantized cache paths. The tier-2 *recall scan* is a separate, self-contained set of fused kernels (Section 6.5); it touches the index metadata only, never the attention math or the cache rows.

6 The recall tier (tier 2, the standard configuration)

At long context the tier is necessary, not optional: at $S=512k$ the reference deployment measures a 99.8% per-step trigger rate with end-to-end quality passing — nearly every decode step consults the archive. A port that stops at tier 1 serves a performance floor, not the method.

Eviction discards rows whose importance arrives later. The recall tier keeps every evicted row reachable per step at bounded cost. All thresholds self-calibrate from the prefix; nothing is trained.

6.1 The selection principle: certified competition, and why the fired set is variable

The fired set’s size varies per step because selection is not a quota but a *competition against the query’s own attended-tier optimum*. The complete decision, per head h , per archived row a :

1. **The standard to beat.** $\max_1^{(h)}$ = the best attention score this head’s query achieves over the *kept* rows. This is the only quantity an archived row must exceed to matter: a row scoring below it would receive negligible softmax mass next to what the attended tier already offers. Everything downstream is query-dependent because this standard is.
2. **The gate (skip whole scans).** If the kept-tier softmax is already confident — entropy below θ_g — the answer is in the attended tier and the scan is skipped for this head. θ_g is self-calibrated as the α -quantile of entropy over *hard* calibration queries (those whose true causal-argmax target is archived), with $\alpha = (1 - \tau)/2 = 5\%$ at $\tau = 0.90$, so at most $\sim 5\%$ of known-hard queries are wrongly gated off; if that threshold would pass more than 60% of all queries the gate cannot separate and disables itself ($\theta_g = -\infty$).

3. **The proxy score.** The true score $s q \cdot \tilde{C}_a$ cannot be afforded against every archived row; the index stores an exact part and a sketch: $\text{idx}_a = s (q_{\text{rope}} \cdot \text{side}_a + q_{\text{sk}} \cdot \text{csk}_a)$ — exact on the 64-dim branch, rank- r on the content.
4. **The certificate (why a proxy can be trusted).** The truncated part of the true score is $\langle q_{\perp}, c_{\perp} \rangle$, the inner product of the query’s and the row’s components outside the sketch subspace. By Cauchy–Schwarz it is at most $\|q_{\perp}\| \|c_{\perp}\| = q_{\text{res}} \varrho_a$; scaled by $1/\sqrt{512-r}$ it becomes an expected-magnitude (isotropic) correction rather than the worst case, and z dials how much of it to credit: $\text{score}_a = \text{idx}_a + z s q_{\text{res}} \varrho_a / \sqrt{512-r}$.
5. **z self-calibration (the recall guarantee, in closed form).** The tier has one quality parameter, τ (default 0.90); everything else is derived from it. Split the miss budget $1 - \tau$ evenly between the two stages that can lose a target: the entropy gate may falsely close on a hard query with probability at most $\alpha = (1 - \tau)/2$, so the scan itself must recover at level $\tau_s = \tau/(1 - \alpha)$, giving overall recall $(1 - \alpha) \tau_s = \tau$.

For a hard calibration query q the archived target wins exactly when $\text{idx}_t + z \text{cert}_t > \max_1$, i.e. when $z > z_q := (\max_1 - \text{idx}_t) / \text{cert}_t$. Over n exchangeable hard samples, take the order statistics $z_{(1)} \leq \dots \leq z_{(n)}$ and set

$$z = z_{(k)}, \quad k = \lceil (n+1) \tau_s \rceil.$$

This is the conformal quantile: by exchangeability alone — no distributional assumption on queries or rows — a fresh hard query’s required inflation falls at or below $z_{(k)}$ with probability at least τ_s , so $P(\text{target recovered}) \geq \tau_s$ marginally. The quantile exists iff $n \geq \tau_s / (1 - \tau_s)$ (18 samples at $\tau = 0.9$), which *derives* the calibration sample size: collect decode queries, doubling the window, until that many hard samples exist (or a bounded cap is hit, in which case z clamps to a safety maximum z_{max} and errs toward over-fetching, never under-recall). An earlier implementation searched an 11-rung ladder for the smallest passing z ; the ladder was a discretization of this same quantile that could also undershoot its own target between rungs, and is gone — along with the ladder itself, its fallback rung, the fixed calibration count, and the separate minimum-hard-sample knob, all of which this one formula replaces.

6. **Fire and fetch.** Row a fires iff $\text{score}_a > \max_1^{(h)}$ and the gate is open; the fetch is the union over heads (per-head top- j capped first when the cap is set — capped selection is thus “the top- j among certified winners”, keeping the competition’s precision while bounding volume).

The variability now has a one-line explanation: *the fired count is the number of rows that certifiably might beat what the attended tier already gives this query.* A well-served query fires zero (high $\max_1$, closed gate); a query whose target was evicted fires the few rows that plausibly hold it; at $S=512\text{k}$ nearly every step fires something (the measured 99.8% trigger rate) because at that length the kept tier alone rarely dominates. A fixed quota cannot reproduce this: set below a query’s need it misses targets, set above it fetches rows that lose the competition anyway. The variable set is the smallest set maintaining the calibrated recall target per query; the explicit cap then trades a bounded slice of that guarantee for a hard latency bound (Section 7).

6.2 Index build (per layer; provisional at the first decode step, calibrated asynchronously)

The index is deliberately *not* built at prefill end (under chunked prefill the last chunk is not reliably identifiable from the backend, and the calibration queries do not exist yet). The build has two stages:

1. **Provisional build (first decode step, synchronous).** A conservative index serving from step one: V = the first r rows of the identity (no SVD yet), gate fully open ($\theta_g = -\infty$), $z = Z_{\max}$ — over-fetch, never under-recall. (Cache-row proxy queries were tried and rejected: a row used as its own query has itself as causal argmax, always inside the attended tail, so *no* sample is ever hard and z flaps uncalibrated.)
2. **Calibrated build (asynchronous, off the decode stream).** Real decode queries are collected per layer (window starts at 8 steps, grows by the measured hard rate up to 64); once enough hard samples exist, `RecallTier.build` runs on a side CUDA stream and is installed atomically. A layer calibrated once donates its basis (v_{init}) to later rebuilds — the certificate is a Cauchy–Schwarz bound, sound for *any* row-orthonormal basis, so a re-used basis only loosens, never misses.

Inputs: pool rows $C \in \mathbb{R}^{T \times 576}$ (fp32 for the build), the tier-1 mask keep, and n *calibration queries* $q^{\text{cal}} \in \mathbb{R}^{n \times H \times 576}$ (real decode queries in absorbed form; H heads), with their positions.

1. Archive $\mathcal{A} = \{u : \neg \text{keep}_u\}$; $A = |\mathcal{A}|$.
2. **Sketch basis:** center the calibration queries’ content part and take the top- r eigenvectors of its covariance ($r = 64$ recommended GPU-resident): $V \in \mathbb{R}^{r \times 512} \leftarrow \text{rows of } \text{eigh}((q_{:,512}^{\text{cal}} - \overline{q_{:,512}^{\text{cal}}})^\top (q_{:,512}^{\text{cal}} - \overline{q_{:,512}^{\text{cal}}}))$ with the r largest eigenvalues, in descending order. (Equivalently the top- r right singular vectors; the `eigh` form is what the code ships — measured $17.4 \rightarrow 4.3$ ms at unit subspace alignment, where randomized `svd_lowrank` aligned only 0.87 and was rejected.)
3. **Per-row index entries**, for $a \in \mathcal{A}$: sketch $\text{csk}_a = \hat{c}_a V^\top \in \mathbb{R}^r$; residual norm $\varrho_a = \|\hat{c}_a - \text{csk}_a V\|$; branch $\text{side}_a = r_a \in \mathbb{R}^{64}$. Index footprint per row: $r + 1 + 64$ floats.
4. **Calibration labels:** for each calibration query, find its causal argmax row over the *full* cache (chunk the key axis; the pool is complete by the partition invariant). A query is *hard* iff its target is archived.
5. **Calibration queries must be REAL queries.** The gate and z are calibrated on *hard* queries — those whose causal-argmax target is archived. Standing in cache rows for queries (an attractive shortcut, since a row and an expanded query share the 576-dim space in the absorbed form) is degenerate, not merely biased: a cache row used as its own query has itself as causal argmax, and its own position always lies inside the recent window, which tier 1 always keeps. So *no* sample is ever hard, by construction. Measured on the `sglang` port: $n_{\text{hard}} = 0$ on every build, z falling back to an uncalibrated constant, and firing on 1.31% of the archive against the reference record’s 0.024% — a $60\times$ inflation of the fetch volume, invisible in any output, visible only as latency.

Where an engine can reach real prefill queries, use the last n_{cal} of them. Where it cannot — an attention backend sees MLA prefill on the un-absorbed path, $[\cdot, H, 192]$, and absorbing to the 576-dim index needs W_{kc} , a model weight — calibrate instead on the request’s first n_{cal} *decode* queries, which arrive already absorbed. Recall stays inactive for those steps, which is the price. On the port this restored n_{hard} to 3–63 per layer, per-layer calibrated z landing continuously in $[0, Z_{\max}]$, and firing on 0.021% of the archive — the reference’s regime. Decode queries are arguably the better calibration set regardless: they are the distribution recall actually serves.

6. **Entropy gate:** over kept rows only, per query compute $\text{ent} = H(\text{softmax}(sq C_{\text{kept}}^\top))$. Set the gate threshold θ_g to the α -quantile of ent over hard queries, $\alpha = (1 - \tau)/2$ (only when at

least $\text{min_hard} = \lceil \tau_s / (1 - \tau_s) \rceil = 18$ hard queries exist; otherwise the gate stays open); if the resulting gate would pass $> 60\%$ of all queries, disable the gate ($\theta_g = -\infty$). The gate exists to skip scans when the kept-set softmax is already confident.

7. **Certificate weight z (closed form)**.: for hard queries, with $q_{\text{sk}} = q_{:512} V^\top$, $q_{\text{res}} = \|q_{:512} - q_{\text{sk}} V\|$ (equivalently $\sqrt{\|q_{:512}\|^2 - \|q_{\text{sk}}\|^2}$ since V is row-orthonormal), target-row proxy score $\text{idx}_t = s(q_{512} \cdot \text{side}_t + q_{\text{sk}} \cdot \text{csk}_t)$ — computed with the query operands *rounded to their storage dtypes first* (bf16 side, fp16 sketch), so calibration matches the serving kernel bit-for-bit (quantize-then-calibrate) — and certificate $\text{cert}_t = s q_{\text{res}} \varrho_t / \sqrt{512 - r}$, each hard query requires $z > z_q := (\text{max}_1 - \text{idx}_t) / \text{cert}_t$; set z to the conformal order statistic $z = \min(z_{(k)}, Z_{\text{max}})$ with $k = \lceil (n+1)\tau_s \rceil$ over the n hard samples and $Z_{\text{max}} = 8$. If $n < \text{min_hard}$ (or $n = 0$), clamp $z = Z_{\text{max}}$ and keep collecting (over-fetch, never under-recall). (z inflates the proxy by a Cauchy–Schwarz-style bound on the sketch truncation error, so a true target is not missed for lack of sketch rank.)

6.3 Per-step query (each decode step; per layer)

Given the step’s expanded queries $q \in \mathbb{R}^{H \times 576}$:

$$\begin{aligned}
\text{max}_1, \text{ent} &\leftarrow \text{from } s q C_{\text{kept}}^\top && \text{(the kernel’s own LSE pass can supply this)} \\
\text{gate} &\leftarrow \text{ent} > \theta_g && [H] \\
\text{score} &\leftarrow s(q_{512} \cdot \text{side}^\top + q_{\text{sk}} \cdot \text{csk}^\top) + z s \frac{q_{\text{res}} \varrho^\top}{\sqrt{512 - r}} && [H, A] \text{ (one GEMV-shaped scan)} \\
\text{fire} &\leftarrow (\text{score} > \text{max}_1) \wedge \text{gate} && [H, A] \\
\text{fetch} &\leftarrow \text{fire} \wedge \text{topj-cap}(\text{score}) && \text{per head, if topj} > 0
\end{aligned}$$

Fetch the union over heads of fired archive rows and splice them into this step’s attention as one extra split-KV partition. Complexity: one $[H \times A] \cdot (r+65)$ scan per layer per step.

6.4 Fused in-graph implementation (the shipped scan)

The per-step query above is one GEMV-shaped scan *mathematically*; the shipped kernel realizes the whole recall step — prologue, scan, compaction, and CSR pack — as **seven fused Triton kernels baked into the decode CUDA graph itself**, so it costs one `cudaGraphLaunch` per step with no second graph and no VestigeKV-triggered recapture:

1. **prologue (2 kernels)**: an online-softmax pass over the kept rows (forked from the platform’s decode-attention accumulation) supplies max_1 and the entropy gate; a split-NK layout restores occupancy (one CTA per (pair, head-block)), and the merge kernel folds in the $q_{\text{sk}}/q_{\text{side}}/q_{\text{res}}$ projections. Queries are read in place from the stacked `qbuf`; no materialized gather.
2. **scan (1 kernel)**: native bf16/fp16 tensor-core dots with fp32-ieee accumulation over a worst-case-capacity pack; each program covers one 1024-row compact bucket and emits its fired count as a plain store.
3. **compaction (2 kernels)**: an exclusive prefix over buckets (self-zeroing for the next replayed step) then an ordered fired-row write — deterministic by construction, so the fired layout is identical across replays (a bit-parity requirement).

4. **CSR pack (2 kernels)**: forked from the platform’s kv-indices builder; a single-program-per-layer prep applies the step’s append and writes indptr from pre-step state only (no cross-program visibility race), then a (layers $\times$ lanes) gather streams kept+-fired rows into the CSR the attention kernel reads.

Correctness rests on three invariants: (i) the pack and every baked buffer are sized for the worst case at startup, so contents refresh in place and the model graph never recaptures for VestigeKV reasons; (ii) placeholder pairs and padded lanes carry zero lengths and point at a trash slot, so the always-running kernels fire nothing and write where nothing reads — and the grid is clipped per batch-size class to the live pair count, so placeholders are not even launched; (iii) the scan uses the *previous* step’s query (stale-by-one), so it can run at the top of the captured step before any layer overwrites `qbuf`. Every kernel retains its pre-fusion implementation in-tree as a bit-parity oracle, and the fused pipeline’s 2048-token greedy output is token-identical to the unfused one. A kill-switch env var falls back to a separately captured scan graph.

6.5 How the per-step recall count arises

The most-asked implementation question is how the number of recalled rows is computed. The precise answer: it is not computed — it *emerges* as the cardinality of a union of per-head threshold sets,

$$N_{\text{step}} = \left| \bigcup_{h: \text{gate}_h} (\{a \in \mathcal{A} : \text{score}_{h,a} > \max_1^{(h)}\} \cap \underbrace{\text{top-}j_h}_{\text{optional clip}}) \right|,$$

produced by a six-stage pipeline per layer per step (shapes in brackets): (1) kept-tier scores give per-head $\max_1$ and entropy $[H]$ (the kernel’s LSE pass supplies both); (2) the gate selects the scanning heads, $H' = |\{h : \text{ent}_h > \theta_g\}|$ — the first source of variation; (3) one GEMV-shaped scan scores $[H' \times A]$; (4) thresholding against each head’s own $\max_1$ yields per-head crossing counts F_h — the second source; (5) the cap clips each head to $\min(F_h, j)$; (6) the union over heads deduplicates — the third source (head overlap).

There is deliberately no closed-form count: N_{step} is the measure of the score distribution’s level set above a query-dependent threshold — data, not configuration. What *is* configured is the distribution’s shape, through exactly two knobs. The calibration target (via z , the conformal quantile of required inflations at the 90% recall criterion) sets the operating point, making N_{step} the size of the smallest certified set at that point — one row more is waste, one fewer breaks the guarantee; raising the target shifts the whole distribution up. The cap then clips the tail at $j \times H'$.

Measured shape at $S=512k$ (reference record): trigger rate 99.8% of steps ($N \geq 1$), mean fetch 0.024% of the archive (≈ 120 rows), uncapped worst observation 3,983, capped worst 296. The distribution is frequent-and-small with a heavy tail — which is precisely what the cap exists to clip, and how a serving SLO should quote it (mean plus hard bound). A port should expose an N_{step} histogram counter from day one; it is the single most informative health signal of the tier.

6.6 Is tier 1 still necessary at long context, and does recall depend on it?

Yes. The 99.8% trigger rate is a miss *frequency*, not a workload share: at $S=512k$ tier 1 supplies over 99% of the attended rows ($\approx 16,384$ kept vs ≈ 120 fetched per step), so tier 1 is the hit path and recall the miss path of a cache hierarchy. Recall is also defined in terms of tier 1 three times over (the fire criterion competes against the kept-tier maximum, the entropy gate reads the kept-tier softmax, and calibration defines hardness relative to the partition); the no-competition

counterfactual collapses to similarity top- k (recall@4 = 0.489 or 99.5% fire rate). Full argument: the defense document (`docs/defense/defense.tex`, Q1).

6.7 Relation to ArkVale

ArkVale (Chen et al., 2024) is the closest prior design — evict, summarize, recall on demand. This tier departs from it on every axis that matters: individual *rows* with an exact 64-dim summand plus a rank- r sketch and a residual certificate, rather than lossy per-*page* bounding digests (a digest cannot be tight under RoPE — measured layer-uniformity 0.672 NoPE vs. 0.024 RoPE — so ArkVale’s regime is structurally the loose one); admission by *certified competition* against the query’s own kept-tier maximum with a self-calibrated guarantee, rather than digest-similarity ranking; and query-*independent* eviction, so the partition exists before any query does. The two designs also degenerate in dual directions under pressure (ArkVale: quality, as pages fall off the budget; here: worst-case attended-set size, never quality, since fetched rows are exact), and every recall-path optimization above is NoPE-enabled with a RoPE counterfactual. The full five-point comparison, the NoPE enablers, memory placement (no paging), and the duality argument: the defense document (`docs/defense/defense.tex`, Q2).

6.8 The fetch cap: an optional deployment optimization (not part of the algorithm)

The serving default is uncapped ($\text{topj} = -1$): fetch the whole fired set. A cap must be *stated explicitly* by the operator; the recommended value is $\text{topj} = 16$, which buys the bounded-fetch guarantee

$$\text{fetch} \leq \text{topj} \times H \text{ rows} / \text{step} / \text{layer}$$

(worst-case observed fire 3,983 rows $\rightarrow$ capped 296; a $13\times$ tightening) at measured-zero quality cost: generation bit-identical, $\Delta\text{bpb} = 0.04\text{ pp}$ inside the pre-registered neutral band, needle recovery identical. Rationale for explicitness: the cap is the one knob trading recall completeness against a worst-case latency/PCIe bound, and the engineer setting it should know which side they are buying.

7 Parameters

One quality knob. The deployment exposes exactly one quality parameter, the recall target $\tau = 0.90$; the rest of the trigger calibration is a closed-form chain, not tuned:

$$\alpha = \frac{1-\tau}{2}, \quad \tau_s = \frac{\tau}{1-\alpha}, \quad z_p = z_{(k)} \text{ with } k = \lceil (n+1)\tau_s \rceil, \quad \text{min_hard} = \lceil \frac{\tau_s}{1-\tau_s} \rceil = 18,$$

plus an $8 \rightarrow 64$ adaptive calibration window and a $Z_{\text{max}} = 8$ clamp.

Structural constants (not tuned per deployment; each with a stated reason):

constant	value	rationale / provenance
ρ	1/32	compression ratio; the only ratio measured flat in context length (an experimental axis, not a runtime knob)
κ	16	low-pass bandwidth of Eq. (1); counts frequency bins, so a fixed block window keeps its meaning scale-invariant
CLOSE_BLOCK	4096	the σ transform window and close granularity; anchors κ 's scale invariance and makes σ incremental
r (sketch)	64	index rank; minimizes per-step scan bytes (the archive is GPU-resident, so fetches are free and larger r only buys a lower fire rate)
sinks	4	always-kept head rows
unclosed tail	< 4096	the always-attended remainder since the last close; absorbs the recent-window role (a fixed $W=256$ window survives only as a debug checker's scale)

Optional optimization. `topj` (fetch cap) defaults to -1 (uncapped, the deployed form); a bounded fetch (16 recommended) is an optional worst-case-latency optimization, adopted from ArkVale’s budgeted admission and applied to the certified scores.

Operational values (partition, prefill chunking, backends, transport tuning) are deployment-specific; the reference deployment records them, one file, each entry carrying its value, rationale, and provenance, in `config/recommended.json` of the experiment repository. **Fool-proof rule:** a default may do the whole thing or nothing, never a fraction nobody typed — which is why the cap defaults off and algorithm knobs are typed explicitly at launch.

8 Decode-time block close and rebalance

Tier-1 selection is not a one-shot prefill event. The decode loop closes one block per $B = 4096$ decoded tokens (`while seq_len - closed >= B`, catching up multiple blocks if needed); each close, per (request, MLA layer):

1. compute σ for the newly closed block (Eq. (1), fused projection form) and append it to the immutable per-block record;
2. **global rebalance:** re-run Eq. (2) over all closed rows with the grown quota $m = \max(1, \text{round}(\rho \cdot \text{closed}))$ — rows move in either direction (a kept row whose σ no longer makes the cut is demoted to the archive);
3. rewrite the kept table as [selected closed rows | unclosed tail]; the tail then grows one row per step until the next close;
4. **refresh the recall index membership only:** extend the closed-prefix operand caches (sketch/residual tables are append-only over closed rows) and re-derive the archive/kept index selection. The calibrated thresholds (z, θ_g) are *not* refit — the conformal certificate is sound for any archive membership, so it survives rebalance.

The close runs synchronously off the captured graph (~ 1 ms per 4096 steps, amortized to noise), and bumps a pack epoch that makes the in-graph buffers re-sync in place (contents, never addresses — the decode graph is not recaptured for a close).

9 Cost model and validation targets

Throughout, k denotes 1024 tokens (4k=4096, 64k=65536, 512k=524288).

Per-token attention volume is not a fixed budget — a property classic sparse attention does not have, so it must enter the port’s latency contract explicitly. The tier-1 attended count is *proportional* ($m + |\text{tail}|$ with $m = \text{round}(\rho T)$; a constant- m schedule exists via `m_fixed` but the proportional schedule is the only one measured quality-flat in context length), and the recall tier makes the per-step count *query-dependent*: the fired set ranges from zero to a measured worst case of 3,983 rows. The algorithm itself is uncapped; the optional cap hard-bounds the per-step fetch at $\text{topj} \times H$ (the non-degeneration guarantee) as a deployment optimization. Uncapped, the theoretical worst case is full fire: that step attends the entire cache, i.e. naive MLA plus the scan overhead (quality unharmed — the fetched rows are exact; steps are independent); the measured worst fire of 3,983 rows is an observation, not a bound. Kernel consequence: size the launch grid for the maximum and rely on empty-split correctness (Section 5); never re-derive the grid from the per-step count in graph mode.

All validation targets in this section are *tier-1 ablation* figures. They are an ablation floor, not a deployable baseline: without the recall tier, quality collapses at long context ($S \gtrsim 256k$), so tier-1-only throughput must never be quoted as the configuration recall is “slower than”. The only legitimate speed comparison is recall-on against the uncompressed baseline.

Honest cost model, and where the speedup lives. Decode step time decomposes as

$$t(S) = C_0 + T_{\text{net}} + A(S),$$

with C_0 the context-independent remainder (MoE experts, linear-attention layers, sampling, host), T_{net} the pipeline-parallel transfer cost (zero on a single device), and $A(S)$ the attention term — the ONLY term this method can shrink. Under the dense baseline $A_{\text{dense}}(S) = k S$ with k the per-row read cost (1152 B/row at bf16). Under VestigeKV,

$$A_{\text{vk}}(S) = \underbrace{k \rho S}_{\text{kept attention}} + \underbrace{k_s(1 - \rho)S}_{\text{archive scan}} + R,$$

where k_s is the scan’s per-row cost (260 B/row with the shipped 16-bit index: side bf16, sketch fp16, residual scalar fp32) and R a small fixed residual (the fused seven-kernel pipeline’s per-step engineering adder, $\sim 68 \mu\text{s}$ by kernel-level profiling, batch-independent). Three consequences, stated plainly:

1. **The advantage is speed, not memory.** Both tiers are GPU-resident and the partition invariant deletes no row: the full latent pool stays in device memory and the wrapper adds index buffers on top. Memory savings would require host-offloading the archive (the $r=192$ variant of Section 7), which this implementation does not do.
2. **The S -slope does not vanish; it shrinks by $\rho + k_s/k \approx 0.26$ with the shipped 16-bit index.** The scan must touch every archived row every step — that is what the recall guarantee costs — so the method cuts the attention slope to roughly a quarter rather than eliminating it. (The fire-set stability gate that licensed the 16-bit scan operands has since passed; see Section 7.)
3. **Short-context speedup is bounded by attention’s share of the step.** On the reference deployment (two nodes over ethernet, $T_{\text{net}} \approx 2.0 \text{ ms}$ measured, $C_0 \approx 5.5 \text{ ms}$, $A_{\text{dense}} \approx 0.57 \text{ ms}$

at $S=64k$ by measured-slope calibration), attention is 7% of the step at 64k, so no selection scheme — this one or any other — can win much there. Below $\sim 32k$ the method is strictly at a disadvantage: the recall pipeline’s fixed cost remains while its saving vanishes, which is exactly the regime the optional dense fallback (above) exists to skip in production. The reported curves run with that fallback disabled, so the left edge of every figure shows this cost undisguised.

This is a long-context method. Measured on the reference deployment (final fused build, $bs=1$ continuous decode), the vestigekv step-time slope is ~ 0.5 ns/token against dense 6.3 ns/token; the crossover sits at $\sim 48k$ and the end-to-end ratio grows with S : $1.07\times$ (128k), $1.20\times$ (272k), $1.39\times$ (496k), extrapolating toward the attention-only asymptote as S grows (the saved term is linear while $C_0 + T_{\text{net}}$ is constant). The measured slope is *below* the byte account because the dense arm’s own long- S slope exceeds its bandwidth floor (split-count growth and length-scaled kernel inefficiencies), which vestigekv sidesteps by holding its attended set near ρS . The crossover is deployment-dependent in the honest direction: the reference testbed’s C_0 (MoE with 3B active parameters) and T_{net} (ethernet pipeline) are LARGE, which suppresses the ratio; a single-device or NVLink deployment moves the crossover earlier and lifts every entry. Each arm’s curve is collected within a single launch so the fixed launch-state terms cancel; cross-arm claims read the same statistic (median per 4k-token bucket) on both, and above $\sim 240k$ the server-side decode log is the per-token authority.

Throughput (64k prefill + 4k decode, batch sweep): the attention share of the step grows with batch (attention cannot batch while the shared weights amortize), so the vestigekv advantage grows with it, against the dense `triton` baseline: within launch-noise of even at $bs=1$ (attention is a few percent of the step there), then +7.7% at $bs=8$ and +8.3% at $bs=12$. The per-step engineering adder of the fused pipeline is $\sim 68 \mu s$ (kernel-level profiling), batch-independent.

10 Validation protocol

Four gate groups, each run before any number is trusted (full detail and thresholds: the defense document, `docs/defense/defense.tex`, Q3):

1. **Plumbing parity:** kill-switch (wrapper disabled) and full-arm (selection keeps *all* rows) outputs must match stock call-for-call / bit-for-bit; every fused kernel keeps its pre-fusion implementation as a bit-parity oracle, and the fused pipeline’s greedy output must be token-identical to the unfused one.
2. **Selector equivalence:** σ and keep must match the reference implementation — bit-exactly on the fallback path, decision-exactly (identical keep masks) for the fused kernels (`engine/test/manual/test` self-contained).
3. **Quality gates** (uncapped, $\rho=1/32$): needle retrieval 1.00 at $8\times-32\times$ / 8k–65k; teacher-forced Δbpb vs the uncompressed engine $\leq 0.15\%$; recall at 1/128 compression reaching 1.00 in the serving loop with the $\sim 512k$ necessity signature (99.8% trigger rate) reproducing.
4. **Regression-net discipline:** every test’s guard must be demonstrated to *fail* on the broken form before it is trusted; the 16-bit scan index additionally passes a fire-set stability gate (Jaccard ≥ 0.99 vs the fp32 arm, each arm calibrated independently).

11 Observed defect classes and mitigations

The port’s postmortem surfaces twelve recurring defect classes; the four structural ones are slot-keyed state going stale on slot reuse (a re-used slot must not inherit the previous request’s kept tables), selection work landing inside the decode window (all σ work belongs at prefill / block close), per-request host loops on the step path (vectorized away), and graph-replay reality (a backend’s `forward.*` python does not run on replay — per-step state must live in the fixed-address buffers the metadata hooks touch). The rest are measurement-hygiene classes: hidden device syncs in capture, padded-lane overruns, arch-specific JIT of unrelated ops, configuration-matrix isolation before attribution, host-side cost invisible to microbenchmarks, benchmark warmup / launch-variance discipline, the second-graph launch tax, and the capacity-grid launch floor. Each class, the bug it produced, and the mitigation: the defense document (`docs/defense/defense.tex`, Q4).

Provenance. Selection math and quality numbers: the pre-registered VestigeKV experimental record (PREREG series; RoPE control measured on DeepSeek-V2-Lite). Engineering contract and performance: the production port to standard `sglang` (branch `vestigekv`), dual-GPU pipeline deployment of Kimi Linear 48B-A3B; all changes outside the attention backend are enumerated in that repository’s port audit. This document froze against commit `e44b914a7` (v2: selection-rule, gate-quantile, calibration and close-semantics sections re-frozen against the code; see `DIFFS.zh.md` for the v1→v2 corrections).